Week 3 Problem Set

Written Report: UAPOML

Classification, Regression Trees, Ensemble Methods & Regularisation

Name: Rudresh Kumar
Roll No: 240893

Random Seed: 42 (used throughout)

1 Logistic Regression

1.1 Summary

We built a binary classifier to predict whether a borrower will experience serious financial distress within two years. Logistic regression is a natural choice here because the output is a probability and the cross-entropy loss is convex in the weights, guaranteeing convergence to the global minimum.

1.2 Pre-processing

- **MonthlyIncome:** 29,731 missing values filled with median = 5,400.
- **NumberOfDependents:** 3,924 missing values filled with median = 0.
- **Class distribution:** 93.3% no default, 6.7% default (imbalanced).

Stratified 70/15/15 split applied (Train: 105,000 — Val: 22,500 — Test: 22,500). StandardScaler fitted only on the training set.

1.3 Metrics Table (Test Set)

Metric	Value
Accuracy	0.9338
Precision	0.5743
Recall	0.0386
F1-Score	0.0723
AUC-ROC	0.7150

1.4 Threshold Analysis

F1-maximising threshold: $\tau = 0.1$ ($F1 = 0.3076$). At the default threshold of 0.5, recall is very low (0.0386) because the model is conservative; very few observations are predicted as default. Lowering the threshold to 0.1 increases recall significantly at the cost of precision.

Feature	Coefficient	—Coeff—
NumberOfTime60-89DaysPastDueNotWorse	-3.7978	3.7978
NumberOfTime30-59DaysPastDueNotWorse	1.9976	1.9976
NumberOfTimes90DaysLate	1.9571	1.9571
age	-0.3992	0.3992
MonthlyIncome	-0.3533	0.3533

1.5 Top-5 Features by —Coefficient—

1.6 Written Interpretation

The five most influential features in our logistic regression model are `NumberOfTime60-89DaysPastDueNotWorse`, `NumberOfTime30-59DaysPastDueNotWorse`, `NumberOfTimes90DaysLate`, `age`, and `MonthlyIncome`. The three past-due count features have the largest absolute coefficients, meaning they are the strongest predictors of serious financial distress. `NumberOfTime60-89DaysPastDueNotWorse` has a large negative coefficient (-3.79), which may seem surprising but reflects the model's internal balancing with the other correlated past-due features. Age has a negative coefficient (-0.40), meaning older borrowers are slightly less likely to default; this aligns with the intuition that older individuals tend to have more financial stability and experience managing debt. `MonthlyIncome` also has a negative coefficient (-0.35), confirming that higher income reduces the probability of default, which makes strong financial sense in a credit scoring context.

2 Decision Trees

2.1 Summary

Decision trees are highly interpretable—a doctor or regulator can follow the exact decision path. This question explores how depth and post-pruning affect generalisation. We compare pre-pruning (depth limiting) against cost-complexity pruning (post-pruning).

2.2 Unpruned Tree

Depth: 91 — Train Accuracy: 1.0000 — Test Accuracy: 0.9805. The perfect training accuracy and lower test accuracy is a clear sign of overfitting; the unpruned tree has memorised the training data.

2.3 Depth vs. Accuracy

Best validation accuracy = 0.9805 at `max_depth` = 10.

2.4 Comparison Table

Model	AUC-ROC	F1-Score	Tree Depth
Unpruned	0.9810	0.9806	9
Depth-limited ($d = 10$)	0.9810	0.9806	9
Cost-Complexity Pruned	0.9810	0.9806	9

2.5 Written Analysis

Cost-complexity pruning achieved the same test AUC-ROC as depth-limited pre-pruning in our experiment; both achieved 0.9810. This dataset is small (1,025 records) and relatively clean, so even the unpruned tree generalises well. Pre-pruning stops the tree early and can miss useful splits further down (the horizon effect), while cost-complexity pruning grows the full tree first and prunes back, giving it a more complete view of the data before making any pruning decisions.

3 Regularisation

3.1 Summary

With 79 correlated features, ordinary least squares is unstable. We applied a log-transform to `SalePrice`, dropped 6 columns with $> 40\%$ missing values, imputed remaining NaNs, one-hot encoded categoricals (resulting in 229 features), and standardised numerics. Ridge, Lasso, and Elastic Net add a penalty term that shrinks coefficients and prevents overfitting.

3.2 Results Table

Model	**Best λ**	**Non-zero Coeffs**	**Test RMSE (£)**	R^2
OLS	–	229	25,883	0.7679
Ridge	429.1934	227	27,393	0.8878
Lasso	0.0054	81	26,721	0.8917
Elastic Net	0.0687	74	28,203	0.8877

3.3 Why Lasso Produces Sparse Solutions

Lasso produces sparse solutions because of the geometry of its L_1 penalty region, which is shaped like a diamond (or hyper-diamond in higher dimensions) with corners sitting exactly on the coordinate axes. When the elliptical loss contours expand outward from the OLS solution, they are very likely to first touch this diamond at one of its corners, which forces one or more coefficients to be exactly zero. Ridge regression uses an L_2 penalty region that is a smooth sphere with no corners, so the loss contours typically touch it at a non-corner point, meaning coefficients are shrunk towards zero but never set exactly to zero. In noisy financial datasets with many irrelevant features, Lasso's sparsity is extremely useful because it automatically performs feature selection: it identifies a small subset of genuinely predictive features (81 out of 229 here) and sets all other coefficients to exactly zero, making the model simpler and more interpretable.

4 Locally Weighted Regression (LWR)

4.1 Summary

LWR fits a separate local linear model for each query point by weighting nearby training points more heavily using a Gaussian kernel. The bandwidth τ controls how local the fit is. The implementation uses NumPy only (no scikit-learn).

τ	Test RMSE
0.05	86,273
0.10	85,893
0.30	85,471 (Best)
1.00	86,093
3.00	86,412

4.2 Bandwidth Comparison

4.3 LWR vs OLS

Model	Test RMSE
LWR ($\tau = 0.3$)	85,471
OLS	86,476

4.4 Written Discussion

LWR most closely approximates OLS when τ is largest ($\tau = 3.0$). As $\tau \to \infty$, the Gaussian kernel weights $w_i = \exp\left(-\frac{\|x_i - x_q\|^2}{2\tau^2}\right)$ all approach 1, meaning every training point gets nearly equal weight, making weighted least squares identical to ordinary least squares. Applying LWR to all numeric features introduces two major difficulties:

1. **Computational cost:** We solve a separate linear system for every test point, scaling as $\mathcal{O}(n_{\text{test}} \times n_{\text{train}})$, becoming extremely slow in high dimensions.
2. **Curse of dimensionality:** In higher dimensions, all points become roughly equidistant and the kernel weights become nearly uniform, losing the locality benefit.

5 Random Forests & Feature Importance

5.1 Summary

Fraud detection has severe class imbalance. Random Forests handle this well via `class_weight='balanced'`. We compare two importance measures: MDI (built-in, fast) and Permutation Importance (more reliable but slower).

5.2 Pre-processing

Dropped 186 columns with $> 50\%$ missing values (out of 394). Remaining: 207 features. Stratified 70/15/15 split: Train: 35,000 — Val: 7,500 — Test: 7,500.

5.3 `max_features` Tuning (5-fold CV)

max_features	Mean AUC-ROC
sqrt	0.8693
log2	0.8694
0.3 (Best)	0.8763
0.5	0.8705

5.4 Test Set Results

OOB Score: 0.9737 — Test AUC-ROC: 0.8936 — Test AUC-PR: 0.4859

5.5 Written Analysis

Looking at the two feature importance plots, the MDI and Permutation Importance rankings do not fully agree with each other. MDI tends to overrate high-cardinality features such as card numbers and transaction IDs because they offer more potential split points, making them appear more important than they actually are. Permutation importance is more reliable since it directly measures the actual drop in AUC-ROC when a feature is randomly shuffled. AUC-PR is more informative than AUC-ROC under class imbalance because AUC-ROC can appear artificially high due to the large number of true negatives (non-fraud cases), while AUC-PR focuses entirely on the minority fraud class and penalises the model for missing actual fraud cases. The OOB error curve plateaus around `n_estimators` $\approx$ 50–100, indicating that additional trees beyond this point provide diminishing returns.

6 XGBoost & Financial Time Series

6.1 Summary

We predict whether a stock will give a positive 21-day forward return. A mandatory time-based split is used (training on data before 2017-01-01 and testing on 2017 onwards) to avoid look-ahead bias.

6.2 Features Engineered

5-day, 10-day, 21-day rolling returns — 21-day rolling volatility — Volume-price ratio — RSI-14. Target: 1 if 21-day forward return > 0 (58.2% positive class). Evaluation period: 2017-01-01 to 2018-01-08 — Train/test boundary: 2017-01-01 — Survivorship bias: S&P 500 constituents only (delisted companies are absent).

6.3 Hyperparameter Grid Search Results

max_depth	learning_rate	subsample	Val AUC-ROC
3	0.05	0.6	0.5248 (Best)
3	0.05	0.8	0.5246
3	0.01	0.6	0.5245
3	0.10	0.6	0.5242
5	0.10	0.8	0.5238

6.4 Test Set Results

Model	AUC-ROC	AUC-PR
XGBoost (best params)	0.4880	0.6053
Naive Majority Baseline	0.5000	–

6.5 Most Important Feature

RSI-14 contributed most by gain. Economically, RSI measures momentum and overbought/oversold conditions—it captures short-term price reversals which are strong predictors of 21-day forward returns.

6.6 Why Random k-fold is Invalid for Financial Time Series

Random k-fold cross-validation is invalid for financial time series because it randomly shuffles data before splitting, which means training folds can contain observations from after the validation fold in calendar time. This causes look-ahead bias: the model effectively sees the future when it learns, which is impossible in real trading. For example, if a stock had a major earnings surprise in Q3 2016, random shuffling might place that Q3 data in the training set while a Q1 2016 observation lands in the validation fold, causing the model to learn from future events to predict the past. This produces validation AUC scores that are much higher than what the model would achieve on genuinely unseen future data. The correct approach is a temporal walk-forward split: always train on all data up to time T and validate strictly on data after T. This is especially important in the UAPOML context because an overfit model underestimates its own prediction uncertainty, leading to overconfident position sizing and potentially large losses in live trading.

7 Bias-Variance Tradeoff

7.1 Summary

We use bootstrap resampling to empirically estimate how much of each model's test error comes from bias (systematic error) vs. variance (sensitivity to the training sample). Models that are too simple have high bias; complex models that memorise noise have high variance.

7.2 Bias-Variance Summary Table

Model	Bias	Variance	Bias^2+Var	Test AUC-ROC
Logistic Regression	0.17383	0.00364	0.17747	0.7988
Decision Stump	0.18976	0.01396	0.20372	0.7361
Unpruned Tree	0.17436	0.14477	0.31913	0.8018
Random Forest	0.16855	0.00676	0.17531	0.8077
XGBoost	0.19381	0.03073	0.22454	0.7911

7.3 Written Explanation: Bagging vs Boosting

Bagging (Random Forest) reduces variance by averaging predictions from B trees trained on different bootstrap samples. The formal variance of this average is $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, where ρ is the correlation between trees and σ^2 is the variance of a single tree. Random Forest reduces ρ by using random feature subsets at each split, which decorrelates the trees and brings the total variance down significantly. However, bagging does not substantially reduce bias because each tree is still trained on a similarly sized dataset; averaging many slightly biased models still gives a biased average. This is confirmed by our results: Random Forest shows the lowest variance (0.00676) but its bias (0.16855) is similar to Logistic Regression. Boosting (XGBoost) reduces both bias and variance. It reduces bias by sequentially fitting each new tree to the residual errors of the previous ensemble, so each round corrects what the previous model got wrong.

Variance is controlled through built-in regularisation including L_1/L_2 penalties, subsampling, and a small learning rate. This is why XGBoost achieves lower total error than the unpruned tree despite having moderate variance.

8 Capstone: Uncertainty-Aware Portfolio Optimization

8.1 Summary

This capstone question integrates gradient boosting, feature selection, time-series cross-validation, and prediction uncertainty estimation via ensemble disagreement. We predict 21-day forward returns for NIFTY-50 constituents and use ensemble disagreement as a proxy for model uncertainty.

8.2 Feature Engineering

8 features per stock per date: 5-day, 21-day, 63-day rolling returns — 21-day and 63-day rolling volatility — Price relative to 52-week high — Price relative to 52-week low — Volume z-score (21-day). Shape after feature engineering: 238,027 rows × 26 columns.

8.3 Expanding-Window CV Results

n_estimators	max_depth	learning_rate	CV AUC
200 (Best)	3 (Best)	0.10 (Best)	0.5342
200	2	0.10	0.5302
100	3	0.10	0.5289
200	3	0.05	0.5282

8.4 Feature Selection & Model Comparison

Top 5 features by permutation importance: `vol_63d`, `rel_52w_high`, `rel_52w_low`, `ret_5d`, `vol_21d`.

Model	Test AUC-ROC
Full model (8 features)	0.5157
5-feature model	0.5138

8.5 Uncertainty Estimation

$M = 5$ gradient boosting models trained on distinct bootstrap samples. For each test observation, mean predicted probability p_{Y_i} and standard deviation σ_i computed. The scatter plot of p_{Y_i} vs σ_i shows that uncertainty is highest for observations with predicted probabilities near 0.5 (decision boundary), which makes intuitive sense: the model is most uncertain when it is least confident about the direction of the return.

8.6 Reflection Essay: Real-World Deployment Risks

Financial markets are non-stationary, meaning their statistical properties change over time. A model trained on data from 2000–2015 may learn patterns specific to that market regime which completely break down during a crisis like COVID-19 in 2020. This overfitting to historical

regimes is one of the biggest risks of deploying gradient boosting models in live trading. Look-ahead bias is another critical risk. Even though we used expanding-window cross-validation to prevent future data from leaking into training, subtle mistakes in feature engineering can reintroduce it. For example, if the 21-day forward return label is accidentally aligned with the wrong date, the model effectively learns from the future, producing unrealistically high AUC scores that disappear in live deployment. Transaction costs are often ignored in academic backtests but are significant in practice. Our long-short strategy rebalances across NIFTY-50 stocks, incurring brokerage fees, bid-ask spreads, and market impact costs each time. These costs can easily erase the apparent profit, especially for smaller portfolio sizes. Finally, the uncertainty measure σ_i directly informs position sizing in an uncertainty-aware portfolio. When σ_i is high, the models disagree strongly about a stock's direction, indicating low confidence—we should reduce or avoid taking a position. When σ_i is low and p_{Y_i} is strongly positive or negative, we can size positions more aggressively. This connects directly to the UAPOML theme: uncertainty estimation is not just a diagnostic tool but a practical input to capital allocation decisions.